Merge Sort

Ordenamiento por mezcla: explicación detallada del algoritmo y de su visualización con bolos en Graph Maker

1. ¿Qué es y qué hace?

Merge Sort (ordenamiento por mezcla) se basa en la estrategia **divide y vencerás**. En vez de ordenar todo el arreglo de una vez, lo **divide en mitades** una y otra vez hasta que cada pedazo tiene un solo elemento (que por definición ya está ordenado). Después **mezcla** los pedazos de dos en dos, de forma ordenada, hasta reconstruir el arreglo completo.

La clave del algoritmo es la operación de **mezcla (merge)**: dadas dos listas que ya están ordenadas, se puede construir una lista ordenada combinada de forma muy eficiente, mirando solo el primer elemento de cada lista.

Analogía

Tienes dos montones de cartas, cada uno ya ordenado con la menor arriba. Para combinarlos, comparas las dos cartas de arriba, tomas la menor y la pones en el montón final. Repites hasta acabar ambos montones.

2. Funcionamiento paso a paso

Fase de división (recursión)

- Se calcula el punto medio del tramo [izq, der]: medio = (izq + der) / 2.
- Se llama recursivamente a Merge Sort para la mitad izquierda [izq, medio].
- Se llama recursivamente para la mitad derecha [medio + 1, der].
- Caso base: si el tramo tiene un solo elemento (izq >= der) no se hace nada.

Fase de mezcla (merge)

- Se copian las dos mitades (ya ordenadas) en listas auxiliares: izquierda y derecha.
- Se usan tres índices: i para la izquierda, j para la derecha y k para la posición del arreglo original donde se escribe.
- Mientras ambas listas tengan elementos, se compara izquierda[i] con derecha[j]; se escribe el menor en la posición k y se avanza el índice correspondiente y k.
- Cuando una lista se acaba, se copian al final los elementos restantes de la otra.

3. Ejemplo resuelto

Se ordena el arreglo **[5, 3, 8, 1, 4]**. Primero se divide:

```
            [5, 3, 8, 1, 4]
              /           \
        [5, 3, 8]         [1, 4]
          /     \           /   \
     [5, 3]    [8]      [1]    [4]
      /   \
    [5]   [3]
```

Luego se mezcla de abajo hacia arriba, en el mismo orden en que lo hace la app:

Mezcla	Qué ocurre	Arreglo resultante
1	Se mezclan [5] y [3] (posiciones 0 a 1). Se compara 5 con 3: sale el 3 y luego sobra el 5.	[**3, 5**, 8, 1, 4]
2	Se mezclan [3, 5] y [8] (posiciones 0 a 2). 3 y 5 son menores que 8, por lo que el 8 queda al final.	[**3, 5, 8**, 1, 4]
3	Se mezclan [1] y [4] (posiciones 3 a 4). Sale el 1 y luego el 4.	[3, 5, 8, **1, 4**]
4	Se mezclan [3, 5, 8] y [1, 4] (posiciones 0 a 4). Comparaciones: 3 vs 1 sale el 1; 3 vs 4 sale el 3; 5 vs 4 sale el 4. Se acabó la derecha, así que se copian los restantes de la izquierda: 5 y 8.	[**1, 3, 4, 5, 8**]

Observa: las mezclas 1 y 2 no cambian visualmente mucho porque el tramo ya venía casi ordenado, pero el algoritmo igual realiza todo el proceso. La mezcla 4 es la que muestra con claridad cómo se intercalan los dos tramos.

Cómo se visualiza en la aplicación

En la pantalla cada número del arreglo se dibuja como un **bolo de boliche**. El tamaño del bolo es proporcional a su valor: el bolo más pequeño representa el valor mínimo y el más grande el valor máximo. El número aparece dentro del bolo. Se pueden usar entre 1 y 20 bolos, con valores entre 1 y 99.

Los controles de la pantalla son los mismos en los cuatro algoritmos:

- **Slider (barra superior):** elige la cantidad de bolos (1 a 20). Genera la secuencia 1, 2, 3, ... n. Se deshabilita mientras se ordena.
- **Cronómetro:** muestra el tiempo en formato mm:ss.d. Se actualiza cada 100 ms con un Timer y se detiene cuando termina el ordenamiento.
- **Mezclar:** desordena los valores actuales con shuffle().
- **Aleatorio:** genera valores aleatorios entre 1 y 99 (pueden repetirse).
- **Ingresar:** permite escribir los valores a mano, separados por coma o espacio, con validación (solo números, de 1 a 99, máximo 20 valores).
- **Guardar / Cargar:** guarda y recupera listas de valores. El grupo de guardado es compartido, así que lo que se guarda en un algoritmo se puede cargar en los otros.
- **Resolver:** inicia el algoritmo con animación.

- **Velocidad:** el desplegable multiplica las pausas de la animación (y la duración con la que los bolos suben y bajan). Se puede cambiar incluso mientras se ordena.

Variables de estado que usa la animación

- **_desde / _hasta:** los extremos del tramo que se está mezclando en este momento.
- **_actual:** la posición k donde se está escribiendo el siguiente valor.
- **_fijos:** permanece en 0 durante el proceso; al final vale n y todos se pintan de verde.
- **_ordenando:** indica si el algoritmo está corriendo.

Colores en la pantalla Merge Sort

Color del bolo	Significado en la pantalla
Amarillo	Todos los bolos del tramo que se está mezclando (de _desde a _hasta).
Naranja	La posición en la que se está escribiendo el valor (_actual).
Blanco	Bolos fuera del tramo actual.
Verde claro	Todos los bolos, únicamente al terminar.

A diferencia de los otros algoritmos, en Merge Sort **no se intercambian dos bolos**: se *sobrescribe* cada posición con el valor que corresponde, tomado de las copias izquierda y derecha. Por eso verás que los números cambian de lugar de golpe en la zona amarilla. En la mezcla hay una pausa de 120 ms al comparar y de 300 ms al escribir (multiplicadas por el factor de velocidad).

4. Código clave

```
Future<void> _mergeSort(int izq, int der) async {
  if (izq >= der) return;               // caso base
  final medio = (izq + der) ~/ 2;
  await _mergeSort(izq, medio);         // mitad izquierda
  await _mergeSort(medio + 1, der);     // mitad derecha
  await _mezclarTramos(izq, medio, der);
}

// Dentro de _mezclarTramos:
final izquierda = _valores.sublist(izq, medio + 1);
final derecha = _valores.sublist(medio + 1, der + 1);
int i = 0, j = 0, k = izq;

while (i < izquierda.length && j < derecha.length) {
  if (izquierda[i] <= derecha[j]) {
    _valores[k] = izquierda[i]; i++;
  } else {
    _valores[k] = derecha[j]; j++;
  }
  k++;
}
// luego se copia lo que sobre de izquierda y de derecha
```

La condición `izquierda[i] <= derecha[j]` usa "menor o igual" a propósito: ante dos valores iguales se toma primero el de la izquierda, y así el algoritmo es **estable**.

5. Complejidad

Característica	Valor	Explicación
Mejor caso	O(n log n)	Siempre divide y mezcla de la misma forma.
Caso promedio	O(n log n)	Hay log n niveles de división y en cada nivel se procesan n elementos.
Peor caso	O(n log n)	Su rendimiento es muy predecible.
Memoria extra	O(n)	Necesita copias auxiliares (sublist) para mezclar.
¿Estable?	Sí	Gracias al uso de <= al mezclar, los elementos iguales conservan su orden.

Con 20 bolos, por ejemplo, hay unos $\log_2(20)$, es decir, entre 4 y 5 niveles de mezcla, mientras que Selection Sort haría 190 comparaciones.

6. Ventajas y desventajas

Ventajas

- Es muy rápido con muchos datos: O(n log n) garantizado.
- Es estable.
- Rendimiento predecible, sin importar el orden inicial.
- Se presta muy bien a procesamiento paralelo y a ordenar datos que no caben en memoria.

Desventajas

- Necesita memoria adicional O(n).
- Es más complejo de implementar (recursión + mezcla).
- Para listas muy pequeñas puede ser más lento que un Insertion Sort por su sobrecarga.

7. ¿Cuándo usarlo?

Cuando se necesita un ordenamiento rápido y confiable para grandes volúmenes de datos, o cuando importa que sea estable (por ejemplo, ordenar registros por una columna sin perder el orden de otra). Variantes de Merge Sort se usan en las librerías estándar de varios lenguajes.